UPSSITECH — 3A SRI
Programmation Orientée Objet

# Le jeu des Pirates

Rapport de projet Java — Conception UML

**Auteur :** Mohamed Aziz Sahli

Année universitaire 2025–2026

# Table des matières

1. Introduction ..... 3
2. Diagramme de classes ..... 3
3. Diagramme de séquence système ..... 5
4. Diagrammes de séquence détaillés ..... 5
   4.1. Case Vol de Vie ..... 6
   4.2. Case Mine ..... 6
   4.3. Case Roulette Russe ..... 7
5. Description des cases spéciales ..... 8
6. Conclusion ..... 8

# 1. Introduction

Ce rapport présente la conception et l'implémentation du jeu des Pirates, réalisé dans le cadre du cours de programmation orientée objet. Le jeu oppose deux pirates : Jack Le Borgne et Bill Jambe de Bois sur un plateau de 30 cases. Chaque joueur lance deux dés à tour de rôle et avance son pion en conséquence. Le premier à atteindre la case 30, ou le dernier à survivre, remporte la partie.

Le développement suit une architecture séparant strictement la logique du jeu de la couche d'affichage, qui implémente l'interface `IAffichage`. Les techniques mises en œuvre incluent l'héritage, les classes abstraites, les interfaces et les énumérés.

# 2. Diagramme de classes

Le diagramme de classes ci-dessous modélise l'ensemble des entités du projet ainsi que leurs relations : héritage, associations et dépendances.

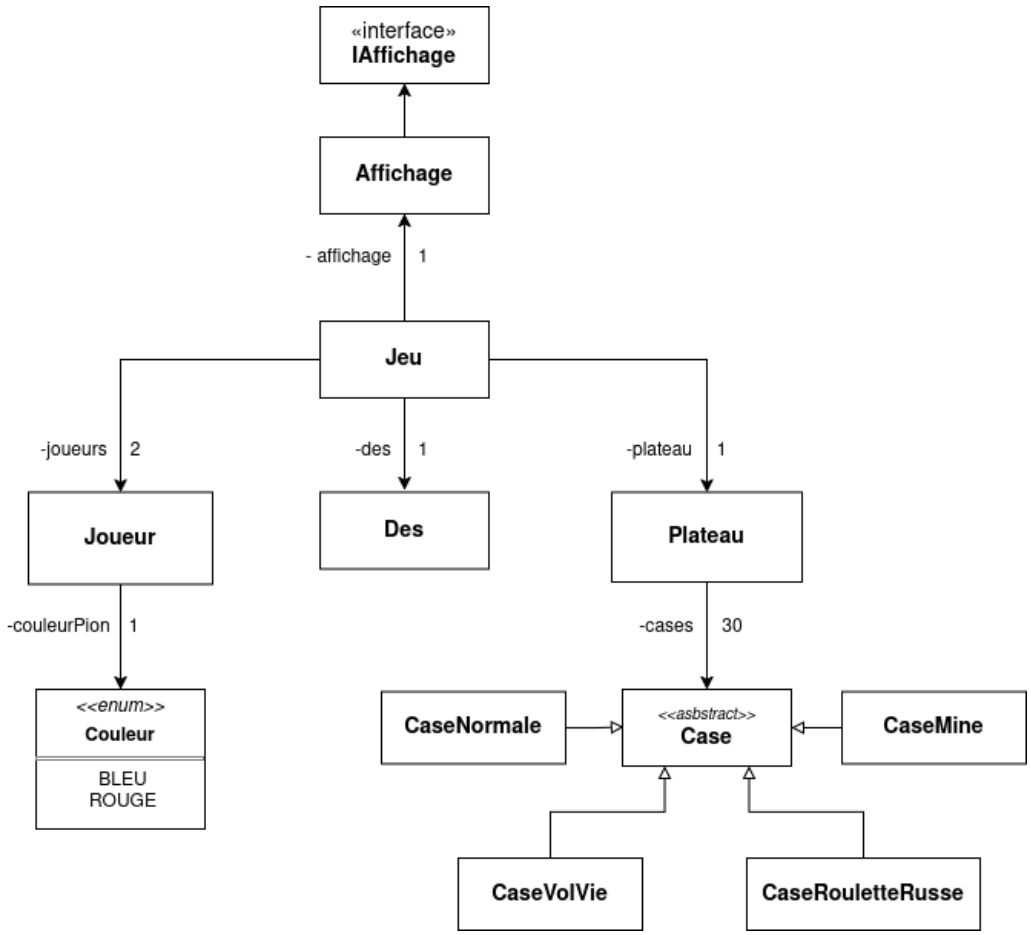


Fig. 1. – Diagramme de classes simplifié

Les points clés de l'architecture sont les suivants :

- La classe abstraite `Case` est étendue par `CaseNormale`, `CaseVolVie`, `CaseMine` et `CaseRouletteRusse`.
- L'interface `IAffichage` est implémentée par la classe `Affichage`, ce qui découple totalement l'affichage du modèle.
- La classe `Jeu` organise la partie en s'appuyant sur `Plateau`, `Des` et un tableau de `Joueur`.
- L'énuméré `Couleur` identifie le pion de chaque joueur (ROUGE ou BLEU).

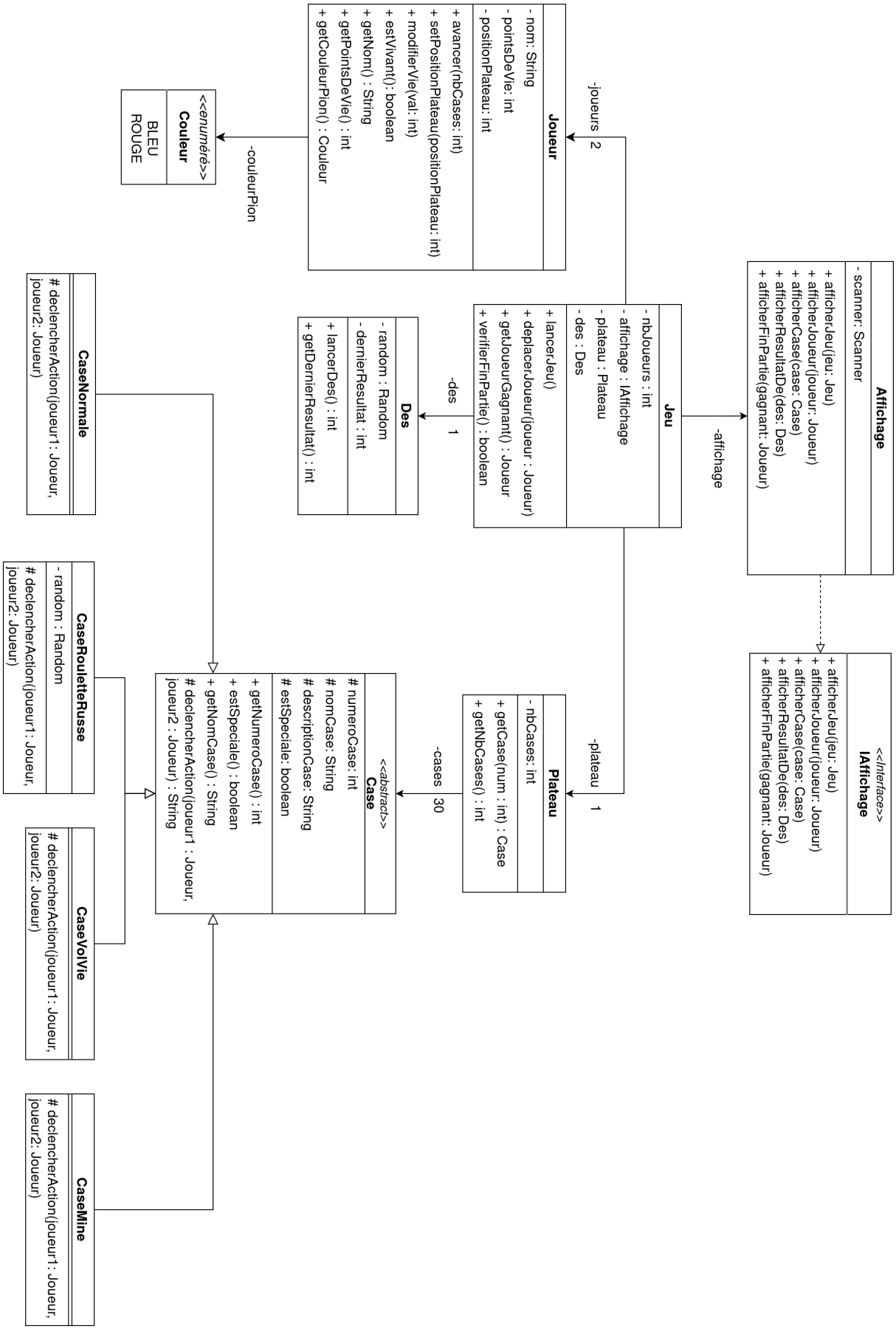


Fig. 2. – Diagramme de classes

## 3. Diagramme de séquence système

Le diagramme de séquence système décrit les interactions entre l'utilisateur et le système lors du déroulement d'un tour de jeu, sans entrer dans le détail des classes internes.

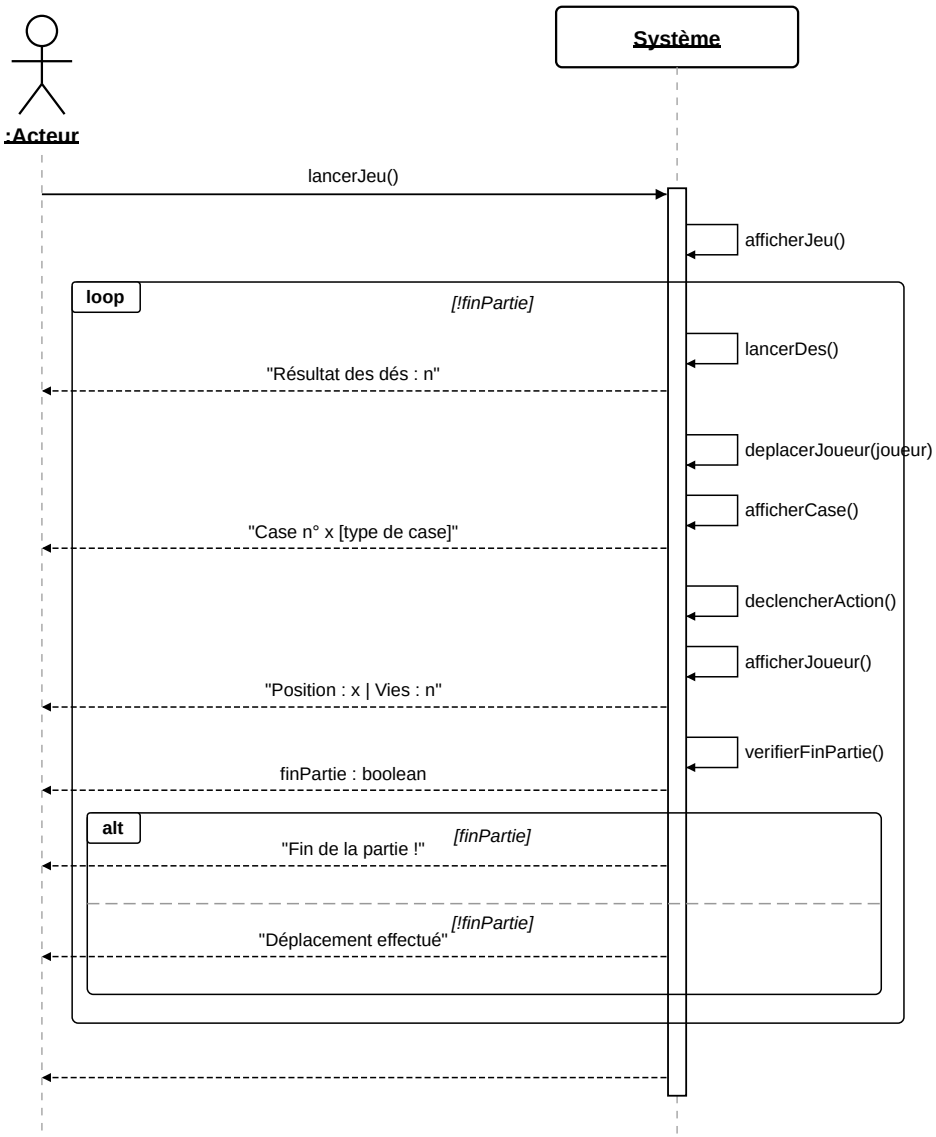


Fig. 3. – Diagramme de séquences système

Le scénario principal est le suivant : le système lance les dés, déplace le joueur actif, déclenche l'action de la case atteinte, puis vérifie les conditions de fin de partie. Ce cycle se répète jusqu'à ce qu'un joueur gagne.

## 4. Diagrammes de séquence détaillés

Cette section présente les diagrammes de séquence détaillés pour les cases spéciales. Ces diagrammes montrent les appels entre objets à l'intérieur du système.

## 4.1. Case Vol de Vie

Lorsqu'un joueur atterrit sur une case Vol de Vie (cases 7 et 21), il gagne un point de vie et son adversaire en perd un.

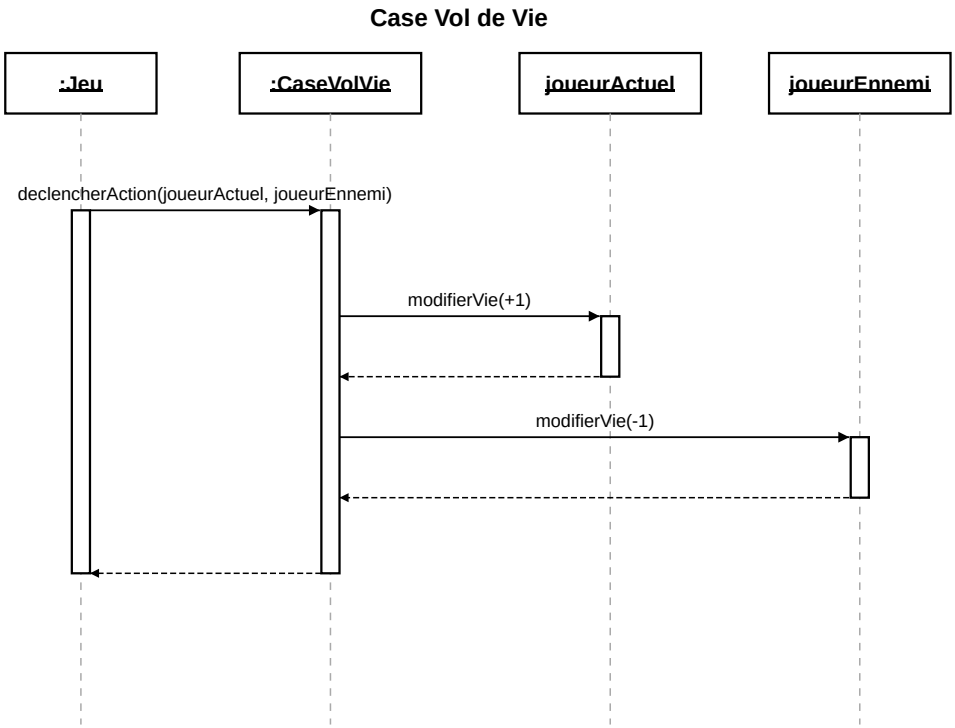


Fig. 4. – Diagramme de séquence détaillé — Case Vol de Vie

## 4.2. Case Mine

Lorsqu'un joueur atterrit sur une case Mine (cases 11 et 19), il perd deux point de vie. Si ses points de vie tombent à zéro ou en dessous, la partie se termine.

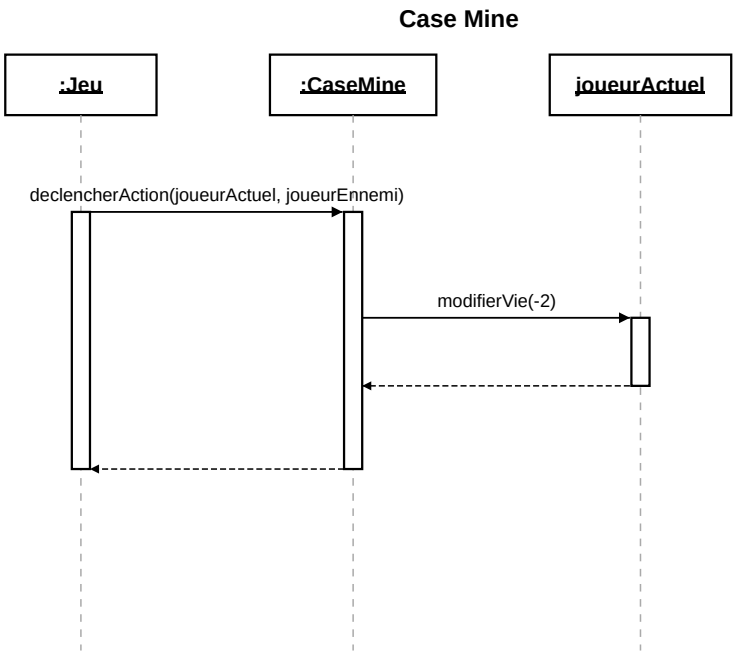


Fig. 5. – Diagramme de séquence détaillé — Case Mine

## 4.3. Case Roulette Russe

Lorsqu'un joueur atterrit sur une case Roulette Russe (cases 13 et 27), sa position est remplacée par un entier aléatoire compris entre 1 et 30.

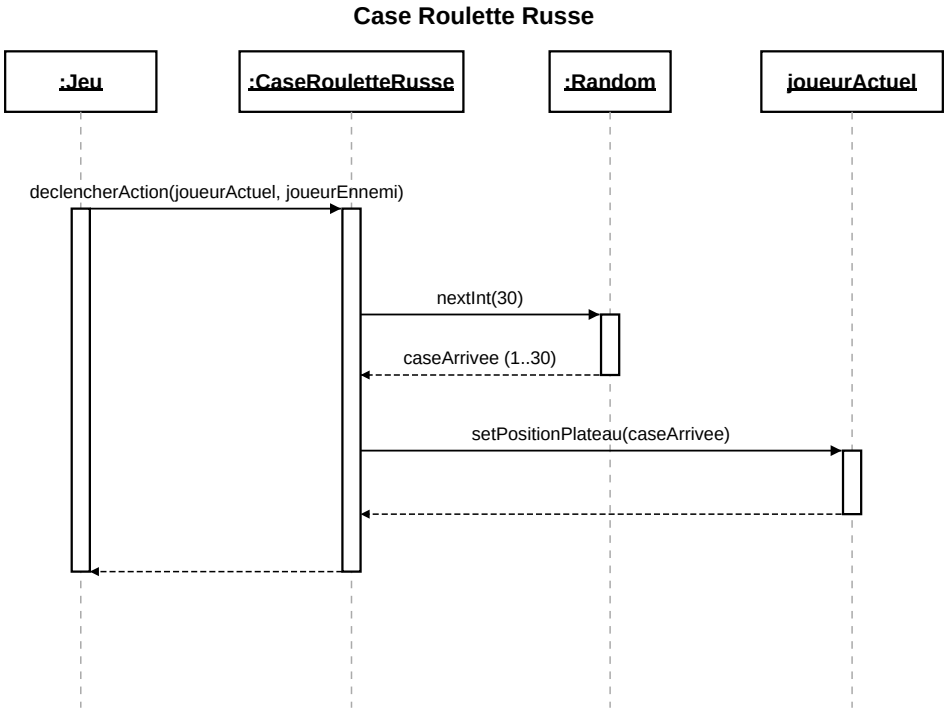


Fig. 6. – Diagramme de séquence détaillé — Case Roulette Russe

## 5. Description des cases spéciales

| Cases | Type | Effet |
|---|---|---|
| 7, 21 | Vol de Vie | +1 PV au joueur actif, −1 PV à l'adversaire |
| 11, 19 | Mine | −2 PV au joueur actif |
| 13, 27 | Roulette Russe | Téléporte le joueur sur une case aléatoire entre 1 et 30 |

## 6. Conclusion

Ce projet a permis de mettre en pratique les principes fondamentaux de la programmation orientée objet : encapsulation, héritage, polymorphisme et séparation des responsabilités. L'architecture retenue, inspirée du patron MVC, garantit que toute logique d'affichage reste confinée à la classe `Affichage`, ce qui facilite la maintenance et l'évolution du code.